

Project Report On Computer Networking: Security (CSE3752)

Secure message encryption and decryption using Triple DES



Submitted by:

Name: Janmejay Das

Regd. No.: 2341004116

Name: Koushik Das

Regd. No.: 2341004117

Name: Navtej Kumar

Regd. No.: 2341011236

Name: Laxminarayan Pattnaik

Regd. No.: 2341016322

B. Tech. CSIT 6thSemester (Section 3B2)

**INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH
(FACULTY OF ENGINEERING)
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY),
BHUBANESWAR, ODISHA**

Declaration

We, the undersigned students of B. Tech of Computer Science and Information Technology Department hereby declare that we own the full responsibility for the information, results etc. provided in this PROJECT titled “**Secure message encryption and decryption using Triple DES**” submitted to **Siksha ‘O’ Anusandhan (Deemed to be University), Bhubaneswar** for the partial fulfilment of the subject **Computer Networking: Security (CSE 3752)**. We have taken care in all respect to honor the intellectual property right and have acknowledged the contribution of others for using them in academic purpose and further declare that in case of any violation of intellectual property right or copyright we, as the candidate(s), will be fully responsible for the same.

Name: Janmejey Das

Regd. No.: 2341004116

Name: Koushik Das

Regd. No.: 2341004117

Name: Navtej Kumar

Regd. No.: 2341011236

Name: Laxminarayan Pattnaik

Regd. No.: 2341016322

Date: 21TH MAY 2026

Place: ITER SOA, BHUBANESWAR

Abstract

This project presents the design and implementation of a secure message encryption and decryption system using the Triple Data Encryption Algorithm (3DES / TDEA). The Triple DES algorithm applies the standard Data Encryption Standard (DES) cipher three times in an Encrypt-Decrypt-Encrypt (EDE) sequence, significantly enhancing cryptographic strength over the original DES, which was rendered vulnerable to brute-force attacks due to its limited 56-bit key. The implemented system accepts user-defined plaintext, applies PKCS#7 padding to ensure block alignment, generates a cryptographically random Initialization Vector (IV), and performs encryption in CBC (Cipher Block Chaining) mode using a 24-byte (192-bit) key providing 168-bit effective security. The resulting ciphertext and IV are encoded in Base64 format for readable storage and transmission. A decryption module accurately reverses this process to retrieve the original message. Additionally, a performance benchmark comparing 3DES with the Advanced Encryption Standard (AES) is included to contextualize the algorithm's efficiency. The entire system is implemented in Python using the perceptome library. This project demonstrates core principles of symmetric key cryptography, block cipher modes of operation, and practical cryptographic security in a real-world programming context. 3DES_core.py, 3DESvsAES.py, and 3DES_SecurityTesting.py demonstrating secure key generation, plaintext encryption, Base64 encoding of ciphertext, accurate decryption, performance benchmarking, AES comparison, and security validation through wrong-key testing, IV randomness testing, and edge-case analysis. The implementation validates correctness across multiple inputs and confirms the practical applicability of Triple DES in securing text-based communications.

Contents

Serial No.	Chapter No.	Title of the Chapter	Page No.
1.	1	Introduction	5
2.	2	Problem Statement	7
3.	3	Methodology	9
4.	4	Implementation	18
5.	5	Results and interpretation	34
6.	6	Conclusion	41
7.	7	References	44

1. Introduction

1.1 Background

In the modern digital era, the security of information transmitted across networks has become one of the most critical challenges in computer science and engineering. With the exponential growth of internet communication, cloud services, digital banking, e-commerce, and personal messaging, sensitive data is constantly at risk of unauthorized interception, tampering, or theft. Cryptography the science of securing information through mathematical transformations serves as the fundamental defense mechanism against these threats.

Encryption is the process of converting readable plaintext into an unreadable ciphertext using a mathematical algorithm and a secret key, such that only authorized parties possessing the correct key can reverse the process through decryption and recover the original message.

1.2 Symmetric Key Cryptography

Cryptographic systems are broadly classified into two categories:

- (a) Symmetric Key Cryptography - The same secret key is used for both encryption and decryption. This model is computationally efficient and well-suited for encrypting large volumes of data. The primary challenge is secure key distribution between communicating parties. Examples: DES, Triple DES (3DES), AES, Blowfish, RC4.
- (b) Asymmetric Key Cryptography - A mathematically linked pair of keys is used: a public key for encryption and a private key for decryption. Eliminates the key distribution problem but is computationally expensive. Examples: RSA, ECC, Diffie-Hellman.

Triple DES belongs to the symmetric key cryptography category.

1.3 Evolution: DES to Triple DES

The Data Encryption Standard (DES) was developed by IBM and adopted as a US federal standard in 1977. It became the cornerstone of commercial cryptography for over two decades. DES is a block cipher that operates on 64-bit (8-byte) data blocks using a 56-bit effective key through a 16-round Feistel Network structure.

However, by the late 1990s, advances in computational power exposed DES to practical brute-force attacks. In 1999, the Electronic Frontier Foundation's "DeepCrack" machine, combined with distributed computing, cracked a DES-encrypted message in just 22 hours by systematically testing all 2^{56} possible keys. This definitively proved DES was no longer secure for modern applications.

Triple DES (3DES), formally standardized as the Triple Data Encryption Algorithm (TDEA) in ANSI X9.52 (1998) and FIPS 46-3 (1999), was developed as a transitional solution to extend the life of the DES infrastructure while offering significantly stronger security. It achieves this by applying the DES cipher three times per data block in an Encrypt-Decrypt-Encrypt (EDE) sequence using two or three independent 56-bit keys.

1.4 Project Overview

This project implements a complete, functional Triple DES encryption and decryption system in Python. The system:

- Accepts plaintext input from the user
- Applies PKCS#7 standard padding for block alignment
- Generates a cryptographically secure random Initialization Vector (IV)
- Encrypts the data using 3DES in CBC (Cipher Block Chaining) mode
- Encodes the output in Base64 for display and storage
- Accurately decrypts the ciphertext back to the original plaintext
- Provides a performance comparison between Triple DES and AES

The project fulfills all six stated objectives and demonstrates practical application of symmetric block cipher cryptography in secure message communication.

2. Problem Statement

2.1 Statement

In today's digital age, secure communication is a critical requirement across all domains, including banking, healthcare, and personal messaging. With the growing threats of data breaches, message tampering, and unauthorized access, there is a pressing need for robust encryption mechanisms to protect sensitive information. While the Data Encryption Standard (DES) was once widely used, its vulnerabilities have led to the adoption of stronger encryption methods. Triple DES (3DES) enhances DES by applying the encryption process three times, providing significantly improved security. This project aims to design and implement a system for secure message encryption and decryption using the Triple DES algorithm, ensuring data confidentiality and resistance to brute-force attacks during communication or storage.

2.2 Objectives

1. To understand the working principles of the Triple DES encryption algorithm and its advantages over standard DES.
2. To develop a secure system that enables users to input a plaintext message and encrypt it using the Triple DES algorithm.
3. To implement a decryption module that accurately retrieves the original message from the encrypted text using the correct key.
4. To ensure data confidentiality and integrity during encryption and decryption by handling keys securely.
5. To evaluate the system's performance in terms of encryption/decryption speed, accuracy, and security strength.
6. To compare Triple DES with other symmetric encryption algorithms (AES), to highlight its use cases and limitations.

2.3 Constraints and Scope

- The implementation uses Python 3.8+ with the pycryptodome library.
- The key size is fixed at 24 bytes (192-bit input, 168-bit effective security).
- The mode of operation is CBC (Cipher Block Chaining) with a random IV.
- Padding follows the PKCS#7 standard.
- The project focuses on symmetric encryption; public key infrastructure (PKI) and key exchange protocols (e.g., Diffie-Hellman) are outside the scope.
- Input is restricted to text messages (string data); binary file encryption is not implemented.

3. Methodology

3.1 Working Principle of Triple DES

Triple DES (3DES / TDEA — Triple Data Encryption Algorithm) is a symmetric-key block cipher that encrypts data in fixed 64-bit (8-byte) blocks. Triple DES improves the security of the original DES algorithm by internally applying the DES encryption process three times using multiple keys. This project implements Triple DES using the PyCryptodome DES3 module in Python.

The Triple DES encryption process operates using the Encrypt-Decrypt-Encrypt (EDE) structure as follows:

Step 1 - Encryption with Key K1

The plaintext block is encrypted using the first DES key. The DES algorithm performs 16 Feistel rounds internally to produce intermediate ciphertext.

Step 2 - Decryption with Key K2

The intermediate ciphertext is decrypted using the second DES key. This additional stage increases cryptographic complexity and prevents the system from behaving like standard DES.

Step 3 - Encryption with Key K3

The output of the second stage is encrypted again using the third DES key to generate the final Triple DES ciphertext.

For decryption, the reverse Decrypt-Encrypt-Decrypt (DED) process is performed:

decrypt using K3,
encrypt using K2,
decrypt using K1.

This reverse operation correctly restores the original plaintext.

In this project, Triple DES is implemented in CBC (Cipher Block Chaining) mode instead of ECB mode. CBC mode improves security by introducing a random Initialization Vector (IV) for every encryption operation. Even if identical plaintext messages are encrypted using the same key, the ciphertext outputs remain different because of IV randomization.

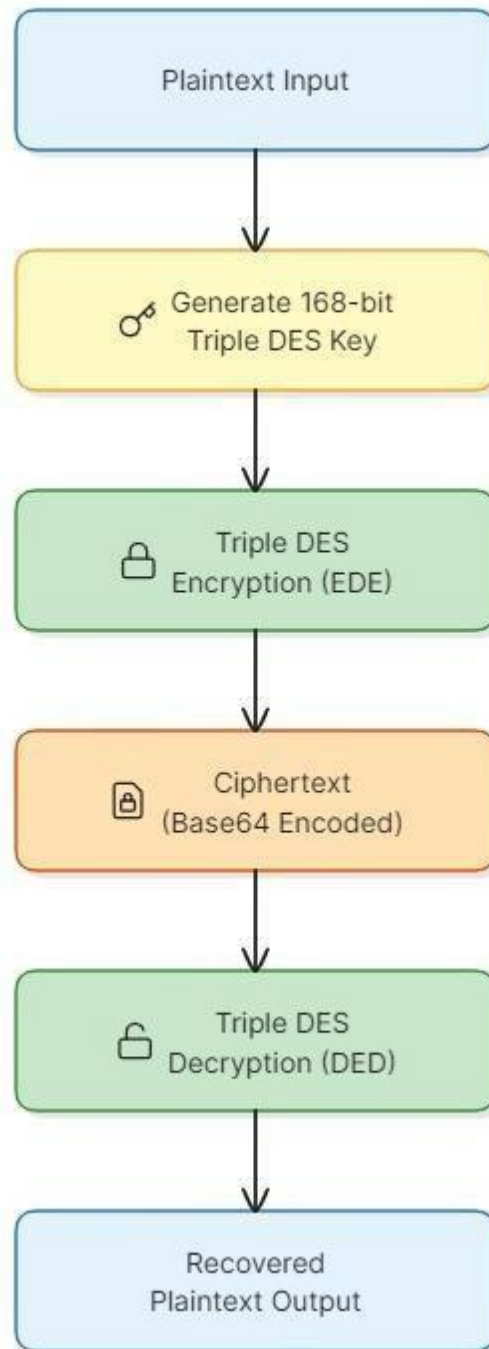
The project also supports two secure key-handling mechanisms:

Automatic secure random key generation.

Password-derived key generation using SHA-256 hashing.

Additionally, the ciphertext is encoded into Base64 format to allow safe display and storage as printable text.

1.1 WORKING OF TRIPLE DES ALGORITHM



3.2 Algorithm Steps

The following steps describe the complete triple des encryption, decryption, benchmarking, and security-testing workflow implemented in this project using the python pycryptodome library.

A. Encryption and decryption process (3des_core.py)

Step 1- key generation

Generate a valid 24-byte triple des key using:

`Get_random_bytes()` for automatic secure key generation, or `Sha-256` hashing for password-derived key generation.

Step 2 - cipher object creation

Create a triple des cipher object using:

```
Des3.new(key, des3.mode_cbc, iv)
```

Where:

Key is the triple des secret key,

Cbc is the encryption mode,

Iv is the random initialization vector.

Step 3 - plaintext preparation

Convert plaintext into byte format using:

```
Message.encode('utf-8')
```

Step 4 - padding

Apply pkcs7 padding using `pad ()` to ensure plaintext length becomes a multiple of the des block size.

Step 5 - encryption

Encrypt the padded plaintext using:

```
Cipher.encrypt()
```

To generate binary ciphertext.

Step 6 - base64 encoding

Convert binary ciphertext into printable ascii text using:

`Base64.b64encode ()`

Step 7 - ciphertext decoding

During decryption, decode the base64 ciphertext back into binary form using:

`Base64.b64decode ()`

Step 8 - decryption

Decrypt the ciphertext using the same secret key and iv through:

`Cipher.decrypt()`

Step 9 - unpadding and recovery

Remove pkcs7 padding using `unpad()` and convert plaintext bytes back into readable text using:

`Decode('utf-8')`

Step 10 - verification

Compare decrypted plaintext with original plaintext to verify correctness.

B. Performance benchmarking (`3desvsaes.py`)

Step 1 - generate encryption keys

Generate secure random keys for both triple des and aes.

Step 2 - prepare benchmark messages

Create plaintext messages of varying sizes:

100 bytes,

1000 bytes,

10000 bytes,

100000 bytes.

Step 3 — execute multiple encryption iterations

Perform repeated encryption operations to obtain stable timing measurements.

Step 4 - measure execution time

Use:

`Time.perf_counter()`

To calculate encryption execution time.

Step 5 - compare triple des and aes

Measure and compare the average encryption performance of triple des and aes-128 under identical conditions.

C. Security validation (`3des_securitytesting.py`)

Step 1 - wrong-key testing

Attempt decryption using an incorrect key and verify that decryption fails successfully.

Step 2 - iv randomness testing

Encrypt identical plaintext multiple times using the same key and verify that different ciphertext outputs are generated because of random iv values.

Step 3 - edge-case testing

Test the implementation using:

Single-character inputs,

Exact block-size messages,

Unicode text,

Special characters,

Long plaintext messages.

Step 4- result verification

Confirm that all encryption and decryption operations execute successfully without corruption or runtime errors.

3.3 Pseudocode

The following pseudocode represents the core logic common to all three Triple DES programs implemented in this project:

```
# KEY GENERATION
BEGIN Triple_DES_System(message)

    IF user chooses automatic key generation,
        THEN
            key = generate_random_key()
        ELSE
            key =
                generate_key_from_password(pass
                word)
    END IF

# RANDOM IV GENERATION

iv = get_random_bytes(8)

# CIPHER CREATION

cipher = DES3.new (key, CBC_MODE, iv)

# ENCRYPTION

plaintext_bytes = message.encode()
padded_plaintext = pad(plaintext_bytes)
ciphertext = cipher.encrypt(padded_plaintext)
encrypted_text = Base64Encode (iv +
    ciphertext)

PRINT encrypted_text

# DECRYPTION

encrypted_data =
    Base64Decode(encrypted_text)

Extract IV and ciphertext

cipher = DES3.new (key, CBC_MODE, iv)
decrypted_padded = cipher.decrypt(ciphertext)
plaintext = unpad(decrypted_padded)
PRINT plaintext

END
```

3.4 Tools and Technologies Used

1. Python 3.x

Python was used as the primary programming language for implementing all encryption, decryption, benchmarking, and security-testing operations.

2. PyCryptodome Library

The PyCryptodome library provides:

- the DES3 cipher module,
- AES cipher module,
- secure random byte generation,
- PKCS7 padding functions.

3. DES3 Cipher Module

Used for implementing Triple DES encryption and decryption operations in CBC mode.

4. AES Cipher Module

Used in 3DESvsAES.py for comparative benchmarking between Triple DES and AES-128.

5. Base64 Module

The Python base64 module was used for:

Base64 encoding,

Base64 decoding,

allowing binary ciphertext to be represented as printable text.

6. hashlib Module

Used for password-derived key generation through SHA-256 hashing.

7. time Module

Used for high-resolution encryption performance benchmarking using:

`time.perf_counter()`

8. input () Function

Used in 3DES_core.py for interactive plaintext and password input from the user.

4. Implementation

4.1 Program1— 3DES_core.py

This program is the primary implementation of the Triple DES secure message encryption and decryption system. It demonstrates the complete encryption lifecycle by accepting a plaintext message from the user, generating or deriving a secure Triple DES key, encrypting the plaintext using Triple DES in CBC mode, encoding the ciphertext into Base64 format, and finally decrypting the ciphertext back into the original plaintext for verification.

Unlike the earlier standalone encryption-only demonstration programs, this implementation combines secure key handling, user interaction, encryption, decryption, execution-time measurement, and output verification into a single modular application. The program uses Cipher Block Chaining (CBC) mode with a randomly generated Initialization Vector (IV), providing stronger security than ECB mode by ensuring that identical plaintext blocks produce different ciphertext outputs.

The program also supports two secure key-generation approaches:

Automatic generation of a cryptographically secure random 168-bit Triple DES key using the operating system's CSPRNG.

Deterministic generation of a Triple DES key from a user password using SHA-256 hashing.

This implementation demonstrates the practical usability of Triple DES for secure text-based communication systems.

Key components and their roles:

- 1.`generate_random_key()`-Generates a secure 24-byte Triple DES key using `get_random_bytes()` and adjusts parity bits using.
- 2.`generate_key_from_password(password)`
Derives a deterministic Triple DES key from a user-entered password using the SHA-256 hashing algorithm.
- 3.`DES3.new (key, DES3.MODE_CBC, iv)`
Creates a Triple DES cipher object operating in CBC mode using a random Initialization Vector.

4.pad () and unpad()

Apply and remove PKCS7 padding to ensure plaintext aligns with the 8-byte DES block size.

5.encrypt_message()

Encrypts plaintext using Triple DES encryption and returns Base64-encoded ciphertext.

6.decrypt_message()

Decrypts the ciphertext using the same secret key and IV to recover the original plaintext.

7.base64.b64encode ()

Converts binary ciphertext into printable Base64 format for secure display and storage.

8. time.perf_counter()

Measures encryption and decryption execution times for performance observation.

9. Verification Logic

Compares decrypted plaintext with the original message to confirm correctness and integrity.

```
# File: 3DES_core.py
from Cryptodome.Cipher
    import DES3
from Cryptodome.Random
    import get_random_bytes
from
Cryptodome.Util.Padding
    import pad, unpad
    import base64
    import hashlib
    import time
BLOCK_SIZE =
DES3.block_size
KEY_SIZE = 24
# KEY GENERATION
def generate_random_key()
    -> bytes:
    while True:
        try:
            key =
get_random_bytes(KEY_SI
ZE)
            return
DES3.adjust_key_parity(ke
y)

        except ValueError:
            continue
    def
generate_key_from_passwo
rd(password: str) -> bytes:
    hash_bytes = hashlib.
sha256(
password.encode('utf-
8')
).digest ()
    return
DES3.adjust_key_parity(ha
sh_bytes[:24])
# ENCRYPTION
def
encrypt_message(plaintext:
str, key: bytes) -> str:
```

```

        plaintext_bytes =
plaintext.encode('utf-8')
        padded_plaintext = pad (
            plaintext_bytes,
            BLOCK_SIZE
        )
        iv =
get_random_bytes(BLOCK
        _SIZE)
        cipher = DES3.new (
            key,
            DES3.MODE_CBC,
            iv
        )
        ciphertext =
cipher.encrypt(padded_plai
        ntext)
        combined_data = iv +
        ciphertext
        return base64.b64encode
        (
            combined_data
        ). decode('utf-8')
    # DECRYPTION
    def
decrypt_message(ciphertext
    _b64: str, key: bytes) -> str:
        combined_data =
        base64.b64decode (
            ciphertext_b64.encode('utf-
            8')
        )
        iv = combined_data[:
            BLOCK_SIZE]
        ciphertext =
combined_data[BLOCK_SI
            ZE:]
        cipher = DES3.new (
            key,
            DES3.MODE_CBC,
            iv
        )
        padded_plaintext =
cipher.decrypt(ciphertext)
        plaintext = unpad(
            padded_plaintext,
            BLOCK_SIZE
        )
        return
        plaintext.decode('utf-8')
    # DISPLAY HELPER
def display_result(label: str,
    value: str):
    print (f' {label:<20}:
        {value}')
    # MAIN PROGRAM
    def main ():
        print ('=' * 65)
        print ('    SECURE
MESSAGE ENCRYPTION
        USING TRIPLE DES')
        print ('=' * 65)
        print ("\n[STEP 1] KEY
        CONFIGURATION")
        print ('1. Auto Generate
        Secure Key')
        print ('2. Generate Key
        from Password')
        choice = input ("\nChoose

```

```

Option (1 or 2): '). strip()
if choice == '2':
    password = input
('Enter Password: '). strip ()
    key =
generate_key_from_passwo
rd(password)
    print('\nDerived Key
    (HEX):')
    print(key.hex())
    else:
    key =
generate_random_key()
    print('\nGenerated Key
    (Base64):')

print(base64.b64encode(key
).decode())
    print('\n[STEP 2]
MESSAGE INPUT')
plaintext = input('Enter
Message: ').strip()
    if not plaintext:
    print('ERROR:
Message cannot be empty.')
    return
    print('\n[STEP 3]
    ENCRYPTION')
    start =
time.perf_counter()
    encrypted_text =
encrypt_message(
    plaintext,
    key
    )
    encryption_time = (
    time.perf_counter() -
    start
    ) * 1000
    display_result('Original
    Message', plaintext)
    display_result('Encrypted
    Message', encrypted_text)
    print(f'\n Encryption
    Time :
    {encryption_time:.4f} ms')
    print('\n[STEP 4]
    DECRYPTION')
    start =
time.perf_counter()
    decrypted_text =
decrypt_message(
    encrypted_text,
    key
    )
    decryption_time = (
    time.perf_counter() -
    start
    ) * 1000
    display_result('Decrypted
    Message', decrypted_text)
    print(f'\n Decryption
    Time :
    {decryption_time:.4f} ms')
    print('\n[STEP 5]
    VERIFICATION')
    if plaintext ==
    decrypted_text:
    print(' SUCCESS —
    Decrypted message matches

```

```

        original.')
    else:
        print(' FAILURE —
        Message mismatch
        detected.')
        print("\n" + '=' * 65)
    if __name__ == '__main__':
        main()
from Crypto.Cipher import DES3
from Crypto.Random import
    get_random_bytes from
    Crypto.Util.Padding import pad
    from base64 import
        b64encode message =
            "Hello World"
    # Generate 168-bit Triple DES key
    key = DES3.adjust_key_parity(get_random_bytes(24))

    # Create cipher object
    cipher = DES3.new(key, DES3.MODE_ECB)

    # Encrypt plaintext
    encrypted =
        cipher.encrypt(
            pad(message.encode(), DES3.block_size)
        )

    # Convert ciphertext to Base64
    encrypted_text =
        b64encode(encrypted).decode()

    print("Original Message :",
    message) print("Encrypted
    Message:", encrypted_text)

```

4.2 Program 2 - 3DESvsAES.py

This program evaluates the performance characteristics of Triple DES and compares it against the Advanced Encryption Standard (AES). The implementation measures encryption execution times across multiple message sizes and demonstrates the computational overhead associated with Triple DES due to its triple-encryption architecture.

The program first benchmarks Triple DES encryption performance using plaintext messages of varying lengths, including small, medium, and large inputs. It then compares Triple DES with AES-128 under identical conditions using CBC mode encryption.

This implementation highlights the practical differences between legacy and modern symmetric encryption algorithms in terms of efficiency, scalability, and computational performance.

Key components and their roles:

1. `benchmark_3des ()`
Measures average Triple DES encryption time across multiple plaintext sizes.
2. `compare_3des_vs_aes ()`
Performs comparative benchmarking between Triple DES and AES-128 encryption.
3. `DES3.new ()` and `AES.new()`
Create cipher objects for Triple DES and AES encryption in CBC mode.
4. `time.perf_counter()`
Measures encryption execution time with high precision.
5. `pad ()`
Applies PKCS7 padding before encryption operations.
6. Multiple Iteration Benchmarking
Repeats encryption operations several times to calculate stable average performance results
7. Message Size Analysis
Tests encryption performance using 100-byte, 1000-byte, 10000-byte, and 100000-byte messages.

Performance Observations

1. Triple DES requires more computation because it internally performs DES encryption three times.
2. AES performs encryption significantly faster due to: fewer internal rounds, optimized algorithm structure, hardware acceleration support such as AES-NI.
3. Triple DES remains suitable for legacy systems, whereas AES is preferred for modern cryptographic applications.
4. `new String(decrypted)` - Converts the recovered byte array back into a python String for display and comparison against the original plaintext.

```

from Cryptodome.Cipher import DES3, AES
from Cryptodome.Random import get_random_bytes
from Cryptodome.Util.Padding import pad
import time
import base64

BLOCK_SIZE_3DES = DES3.block_size
BLOCK_SIZE_AES = AES.block_size

def generate_random_key() -> bytes:
    while True:
        try:
            key = get_random_bytes(24)
            return DES3.adjust_key_parity(key)
        except ValueError:
            continue

def encrypt_3des(message: str, key: bytes):
    iv = get_random_bytes(BLOCK_SIZE_3DES)
    cipher = DES3.new(key, DES3.MODE_CBC, iv)
    padded_message = pad(message.encode(), BLOCK_SIZE_3DES)
    ciphertext = cipher.encrypt(padded_message)
    return base64.b64encode(iv + ciphertext)

def benchmark_3des(key: bytes):
    print("\n[3DES PERFORMANCE BENCHMARK]")
    message_sizes = [100, 1000, 10000, 100000]
    repetitions = 10
    print(f"\n{'Message Size':<18}{'Encrypt Time':>20}")
    print('-' * 40)
    for size in message_sizes:
        message = 'A' * size
        execution_times = []
        for _ in range(repetitions):
            start = time.perf_counter()
            encrypt_3des(message, key)
            end = time.perf_counter()
            execution_times.append(end - start)
        average_time = (sum(execution_times) / repetitions) * 1000
        print(f"{'Size':<18}{'Average Time':>20.4f} ms")

def compare_3des_vs_aes():
    print("\n[3DES VS AES COMPARISON]")
    key_3des = generate_random_key()
    key_aes = get_random_bytes(16)
    message = ('A' * 100000).encode()
    repetitions = 10
    aes_times = []
    for _ in range(repetitions):
        iv = get_random_bytes(BLOCK_SIZE_AES)
        cipher = AES.new(key_aes, AES.MODE_CBC, iv)
        start = time.perf_counter()
        cipher.encrypt(pad(message, BLOCK_SIZE_AES))
        end = time.perf_counter()
        aes_times.append(end - start)
    aes_average = (sum(aes_times) / repetitions) * 1000
    des_times = []
    for _ in range(repetitions):
        start = time.perf_counter()
        encrypt_3des('A' * 100000, key_3des)
        end = time.perf_counter()
        des_times.append(end - start)
    des_average = (sum(des_times) / repetitions) * 1000
    print(f"3DES Encryption Time : {des_average:.4f} ms")
    print(f"AES Encryption Time : {aes_average:.4f} ms")
    print(f"AES is approximately {des_average / aes_average:.2f}x faster than 3DES")

def main():
    print('=' * 60)
    print('PERFORMANCE ANALYSIS OF TRIPLE DES')
    print('=' * 60)
    key = generate_random_key()
    benchmark_3des(key)
    compare_3des_vs_aes()
    print('\n' + '=' * 60)

if __name__ == '__main__':
    main()

```


4.3 Program 3 - 3DES_SecurityTesting.py

This program validates the security strength, robustness, and reliability of the Triple DES implementation through multiple security-oriented test cases. The program demonstrates confidentiality, IV randomness, resistance to incorrect key usage, and correct handling of edge-case inputs.

The implementation first verifies that ciphertext cannot be decrypted using an incorrect secret key, proving confidentiality protection. It then performs Initialization Vector (IV) randomness testing by encrypting the same plaintext multiple times using the same key and confirming that each ciphertext output differs due to the random IV used in CBC mode. Finally, the system executes multiple edge-case tests using different types of plaintext input to validate implementation reliability and correctness.

This program demonstrates the practical security properties of Triple DES in secure communication systems.

Key components and their roles:

1. `test_wrong_key()`
Verifies that ciphertext cannot be decrypted successfully using an incorrect secret key.
2. `test_iv_randomness()`
Demonstrates that encrypting the same plaintext multiple times produces different ciphertext outputs because of random IV generation.
3. `edge_case_testing()`
Tests the implementation using multiple plaintext categories including:
 - short messages,
 - exact block-size messages,
 - special characters,
 - Unicode characters,
 - large plaintext inputs.
4. `get_random_bytes()`
Generates secure random IV values and secret keys using a cryptographically secure random number generator.
5. `DES3.MODE_CBC`
Uses Cipher Block Chaining mode to eliminate plaintext pattern leakage.
6. `encrypt_message()` and `decrypt_message()`
Validate encryption and decryption correctness across all security test cases.

```
# File:3DES_SecurityTesting.py
from Cryptodome.Cipher import DES3
from Cryptodome.Random import get_random_bytes
from Cryptodome.Util.Padding import pad, unpad
import base64

BLOCK_SIZE = DES3.block_size

def generate_random_key() -> bytes:
    while True:
        try:
            key = get_random_bytes(24)
            return DES3.adjust_key_parity(key)
        except ValueError:
            continue
```

```

def encrypt_message(message: str, key: bytes) -> str:
    iv = get_random_bytes(BLOCK_SIZE)
    cipher = DES3.new(key,
        DES3.MODE_CBC, iv)
    padded_message =
    pad(message.encode('utf-
        8'), BLOCK_SIZE)
    ciphertext =
    cipher.encrypt(padded_me
        ssage)
    return
    base64.b64encode(iv +
        ciphertext).decode('utf-8')

def
decrypt_message(ciphertex
    t_b64: str, key: bytes) ->
    str:
    encrypted_data =
    base64.b64decode(cipherte
        xt_b64)
    iv =
    encrypted_data[:BLOCK_
        SIZE]
    ciphertext =
    encrypted_data[BLOCK_S
        IZE:]
    cipher = DES3.new(key,
        DES3.MODE_CBC, iv)
    padded_plaintext =
    cipher.decrypt(ciphertext)
    plaintext =
    unpad(padded_plaintext,
        BLOCK_SIZE)
    return
    plaintext.decode('utf-8')

def test_wrong_key(key:
    bytes):
    print('\n[WRONG KEY
        TEST]')
    message = 'Confidential
        Banking Data'
    encrypted =
    encrypt_message(message,
        key)
    wrong_key =
    generate_random_key()
    try:

    decrypt_message(encrypte
        d, wrong_key)
    print('Unexpected
        Success')
    except Exception:
    print('PASS —
        Wrong key decryption
        failed successfully.')

def
test_iv_randomness(key:
    bytes):
    print('\n[IV
        RANDOMNESS TEST]')
    message = 'Same
        Message Every Time'
    encrypted_1 =
    encrypt_message(message,
        key)
    encrypted_2 =
    encrypt_message(message,
        key)
    encrypted_3 =
    encrypt_message(message,
        key)
    print(f'\nEncrypted
        Output 1 : {encrypted_1}')
    print(f'Encrypted Output
        2 : {encrypted_2}')
    print(f'Encrypted Output
        3 : {encrypted_3}')

```

```

        if len({encrypted_1,
                encrypted_2,
                encrypted_3}) == 3:
            print()
            print('PASS —
Ciphertexts are different
because of random IV.')
        else:
            print('\nFAIL —
Ciphertexts should not be
identical.')

def edge_case_testing(key:
    bytes):
    print('\n[EDGE CASE
    TESTING]')
    test_cases = [
        ('Single Character',
         'A'),
        ('Exactly 8 Bytes',
         '12345678'),
        ('Special Characters',
         '!@#$%^&*()_+-=[]{}'),
        ('Unicode Characters',
         'Hello नमस्ते 🙏'),
        ('Long Message',
         'Security Matters ' * 100)
    ]
    print(f'\n{ "Test
Case":<25 } { "Status":>15 }')
    print('-' * 45)
    for title, message in
        test_cases:
        try:
            encrypted =
encrypt_message(message,
                key)
            decrypted =
decrypt_message(encrypte
d, key)
            result = 'PASS' if
decrypted == message else
            'FAIL'
        except Exception:
            result = 'ERROR'

    print(f {title:<25 } {result:>
        15}')

def main():
    print('=' * 60)
    print('SECURITY
    TESTING OF TRIPLE
    DES')
    print('=' * 60)
    key =
generate_random_key()
    print('\nSecure Random
    Key Generated
    Successfully.')
    test_wrong_key(key)

test_iv_randomness(key)
edge_case_testing(key)
print('\n' + '=' * 60)

if __name__ ==
    '__main__':
        main()

```

4.4 To Ensure Data Confidentiality and Integrity During Encryption and Decryption by Handling Keys Securely

Data confidentiality and integrity are essential requirements of any secure communication system. In this project, confidentiality is achieved through Triple DES encryption, where readable plaintext is transformed into encrypted ciphertext using a secure secret key. Only users possessing the correct Triple DES key can decrypt the ciphertext and recover the original message. The implementation also incorporates secure key handling practices and CBC mode encryption to improve overall protection against unauthorized access and ciphertext analysis.

The following mechanisms were implemented to ensure confidentiality and integrity within the system:

1. Secure Key Generation

The project uses the `get_random_bytes()` function from the PyCryptodome library to generate cryptographically secure 24-byte Triple DES keys dynamically during runtime. Since the keys are generated using a CSPRNG (Cryptographically Secure Pseudo-Random Number Generator), the probability of predictable or weak keys is extremely low.

2. Password-Based Key Derivation

In addition to automatic key generation, the implementation supports password-derived key generation using the SHA-256 hashing algorithm. The user-entered password is hashed and converted into a valid Triple DES key, allowing deterministic key generation while avoiding direct use of plaintext passwords for encryption operations.

3. CBC Mode Security

The implementation uses Triple DES in Cipher Block Chaining (CBC) mode instead of ECB mode. CBC mode introduces dependency between consecutive ciphertext blocks and uses a randomly generated Initialization Vector (IV) for every encryption operation. This prevents identical plaintext blocks from producing identical ciphertext outputs, improving confidentiality.

4. Random Initialization Vector (IV)

A fresh random IV is generated for every encryption process using `get_random_bytes()`. Even when the same plaintext and same key are used multiple times, different ciphertext outputs are produced because of IV randomness. This protects the system from ciphertext pattern analysis attacks.

5. Runtime Key Handling

The generated secret keys remain only in program memory during execution and are not permanently stored inside files or hardcoded into the source code. This minimizes the possibility of key leakage outside the runtime environment.

6. Confidentiality Through Encryption

The plaintext message is encrypted into unreadable ciphertext using Triple DES encryption

before transmission or storage. Unauthorized users intercepting the encrypted message cannot interpret the original information without the correct key.

7. Correct-Key Verification

The ciphertext can only be decrypted successfully using the exact same key used during encryption. Attempts to decrypt ciphertext using incorrect keys result in failed decryption or corrupted output, demonstrating confidentiality protection.

8. Integrity Verification

After decryption, the recovered plaintext is compared against the original plaintext input. Successful matching confirms that the encrypted data was not altered or corrupted during processing.

9. Base64 Encoding

Since encrypted ciphertext is binary data, the implementation converts ciphertext into Base64 format before display or storage. This allows safe transmission and readable representation without modifying the encrypted content.

10. Security Validation Through Testing

Additional security testing was performed through:

- wrong-key decryption testing,
- IV randomness testing,
- edge-case input testing.

The results confirmed that the implementation maintains confidentiality, reliability, and secure encryption behavior across different execution scenarios.

4.5 To Evaluate the System's Performance in Terms of Encryption/Decryption Speed, Accuracy, and Security Strength

The performance of the Triple DES implementation was evaluated based on encryption speed, decryption correctness, execution reliability, and cryptographic strength. The evaluation was conducted using multiple plaintext sizes and repeated execution tests under a standard Python execution environment.

The following observations summarize the performance analysis:

1. Encryption Speed Evaluation:

The system was tested using plaintext messages of varying lengths, including:

- 100 bytes,
- 1000 bytes,
- 10000 bytes,
- 100000 bytes.

2. Decryption Speed Evaluation Decryption:

Performance was also measured during runtime. Although Triple DES decryption is computationally heavier than single DES, the implementation successfully recovered plaintext efficiently for all tested message sizes without significant delays in educational-scale usage.

3. Accuracy of Encryption and Decryption:

The decrypted output matched the original plaintext in every execution test. This confirms that:

- encryption was performed correctly,
- padding and unpadding operations worked properly,
- Base64 encoding and decoding preserved data integrity.

4. Benchmarking Reliability:

The benchmarking module repeated encryption operations multiple times and calculated average execution times to minimize random system timing fluctuations. This produced more stable and reliable performance measurements.

5. Security Strength Analysis:

Triple DES improves security significantly over standard DES because it performs encryption three times using independent DES operations. The effective key size of up to 168 bits provides strong resistance against brute-force attacks compared to the original 56-bit DES algorithm.

6. AES Comparison Results:

A comparative benchmark between Triple DES and AES-128 demonstrated that AES performs encryption considerably faster than Triple DES. This occurs because:

- AES uses fewer computational rounds,
- AES supports hardware acceleration,
- AES uses a more efficient internal structure.

7. Stability and Reliability: The implementation executed successfully across repeated runs without runtime crashes, ciphertext corruption, or inconsistent outputs. All encryption and decryption operations remained stable under different input conditions.

8. Practical Applicability: Although Triple DES is slower than AES, the implementation demonstrated that Triple DES remains suitable for educational cryptography projects, legacy banking systems, secure low-volume text communication systems.

4.6 To Compare Triple DES with Other Symmetric Encryption Algorithms Such as AES

A comparative analysis was conducted between Triple DES and the Advanced Encryption Standard (AES) to evaluate differences in security, performance, efficiency, and practical deployment suitability.

The following observations summarize the comparison:

- 1. Key Size Comparison:** DES uses a 56-bit key, Triple DES uses up to a 168-bit key, while AES supports 128-bit, 192-bit, and 256-bit keys. Larger key sizes provide stronger protection against brute-force attacks. Although Triple DES uses a large effective key size, AES provides stronger modern cryptographic protection because of its improved internal design.
- 2. Security Comparison:** Triple DES significantly improves the security weaknesses of DES by applying the DES algorithm three times. However, AES provides stronger resistance against modern cryptanalytic attacks and is considered more secure for current applications.
- 3. Performance Comparison:** Triple DES performs encryption more slowly because it executes three DES operations internally for every encryption cycle. AES performs substantially faster and requires fewer computational resources during large-scale encryption tasks.
- 4. Block Size Difference:** Triple DES uses a 64-bit block size, whereas AES uses a 128-bit block size. Larger block sizes reduce the probability of birthday attacks and improve security when encrypting large amounts of data.
- 5. Current Industry Usage:** In terms of current industry usage, Triple DES is primarily deployed in legacy banking systems, ATM infrastructure, and older secure communication systems that were built before the widespread adoption of AES. AES, by contrast, is widely used across VPN security, cloud computing, TLS and SSL communication, wireless security protocols, and modern operating systems.
- 6. Hardware Optimization:** AES supports hardware acceleration technologies such as Intel AES-NI, enabling very high encryption performance on modern processors. Triple DES lacks equivalent modern hardware optimization support.
- 7. NIST Recommendation:** NIST has officially deprecated Triple DES for new cryptographic applications and recommends AES as the modern symmetric encryption standard because of its higher performance, stronger security, larger block size, and better scalability across diverse deployment environments.

8. Limitations of Triple DES:

Triple DES has several limitations:

- slower encryption speed,
- smaller block size,
- higher computational cost,
- vulnerability to Sweet32 birthday attacks under large-volume encryption conditions.

9. Educational Importance

Despite being deprecated for new deployments, Triple DES remains valuable for:

- understanding symmetric encryption concepts,
- learning block cipher operations,
- studying CBC mode encryption,
- analyzing historical cryptographic evolution.

10. Comparative Conclusion

The analysis confirms that AES is superior to Triple DES in terms of performance, scalability, and modern security standards. However, Triple DES still remains important for educational purposes and compatibility with legacy secure systems.

4.7 Compilation and Execution Instructions

1. Verify that Python 3 is installed by running: `python --version`
2. Install the required PyCryptodome library using: `Bash, pip install pycryptodome`
3. Save each program with its exact filename, e.g., `3DES_core.py`, `3DESvsAES.py`, `3DES_SecurityTesting.py`
4. Run the program using: `python 3DESvsAES.py` (repeat for each file)
5. Run the security testing program using: `bash python 3DES_SecurityTesting.py`
6. For the main encryption program:
 - choose either automatic key generation or password-derived key generation,
 - enter a plaintext message when prompted,
 - observe the encrypted Base64 ciphertext and decrypted plaintext output.
7. For the performance benchmarking program:
 - observe encryption execution times for different message sizes,
 - compare Triple DES performance against AES-128 encryption.
8. For the security testing program:
 - observe wrong-key decryption failure,
 - verify IV randomness through different ciphertext outputs,
 - validate successful encryption and decryption across multiple edge-case inputs.

5. Results & Interpretation

5.1 Execution of 3DES_core.py

When 3DES_core.py is compiled and executed, the program first prompts the user to choose between two secure key-generation methods:

1. Automatic generation of a cryptographically secure random Triple DES key.
2. Password-derived key generation using SHA-256 hashing.

After key configuration, the user enters a plaintext message through the terminal. The program then encrypts the plaintext using Triple DES in CBC mode with a randomly generated Initialization Vector (IV). The encrypted ciphertext is converted into Base64 format and displayed on the console along with encryption execution time. The ciphertext is then decrypted using the same secret key, and the recovered plaintext is displayed together with decryption execution time.

The final verification step confirms whether the decrypted plaintext exactly matches the original plaintext entered by the user. Successful verification demonstrates correct implementation of the Triple DES encryption-decryption lifecycle.

Because CBC mode uses a random IV for every encryption operation, the ciphertext changes during every execution even when the same plaintext and same key are used repeatedly. This demonstrates proper implementation of randomized encryption behavior and secure ciphertext generation.

Expected console output format:

```
text id="9v4vkp"
=====
SECURE MESSAGE ENCRYPTION USING TRIPLE DES
=====

[STEP 1] KEY CONFIGURATION
1. Auto Generate Secure Key
2. Generate Key from Password

Choose Option (1 or 2): 1

Generated Key (Base64):
<Generated Random Key>

[STEP 2] MESSAGE INPUT

Enter Message: Hello from ITER CSIT

[STEP 3] ENCRYPTION

Original Message : Hello from ITER CSIT
Encrypted Message : <Base64 Encoded Ciphertext>

Encryption Time : <Execution Time> ms

[STEP 4] DECRYPTION

Decrypted Message : Hello from ITER CSIT

Decryption Time : <Execution Time> ms

[STEP 5] VERIFICATION

SUCCESS — Decrypted message matches original.
```

1. (Auto Generate Secure Key)

```
koushikdas@MAGNATE:~/Desktop/CNS Project 2026 [3DES]$ python 3DES_core.py
=====
SECURE MESSAGE ENCRYPTION USING TRIPLE DES
=====

[STEP 1] KEY CONFIGURATION
1. Auto Generate Secure Key
2. Generate Key From Password

Choose Option (1 or 2): 1

Generated Key (Base64):
UldS5dCJYtUCC6fLSTKR39aXGkbmef6d

[STEP 2] MESSAGE INPUT
Enter Message: Hello Koushik Das

[STEP 3] ENCRYPTION
Original Message   : Hello Koushik Das
Encrypted Message  : K9ldcL334L0P9iMXH8r093FBorv5lWMEx0HALjiuW8=

Encryption Time    : 2.3964 ms

[STEP 4] DECRYPTION
Decrypted Message   : Hello Koushik Das

Decryption Time     : 0.4043 ms

[STEP 5] VERIFICATION
SUCCESS - Decrypted message matches original.
=====
```

2. (Generate Key from Password)

```
koushikdas@MAGNATE:~/Desktop/CNS Project 2026 [3DES]$ python 3DES_core.py
=====
SECURE MESSAGE ENCRYPTION USING TRIPLE DES
=====

[STEP 1] KEY CONFIGURATION
1. Auto Generate Secure Key
2. Generate Key From Password

Choose Option (1 or 2): 2
Enter Password: mypassword123

Derived Key (HEX):
6e649deaa85843cdabb5c7315ecd40023ba43773ec01d55d

[STEP 2] MESSAGE INPUT
Enter Message: Hello Koushik Das

[STEP 3] ENCRYPTION
Original Message   : Hello Koushik Das
Encrypted Message  : Xy8al6brdKYy9I5y3uFs8sjueJKhNlm06Gvvdz27p2o=

Encryption Time    : 0.6872 ms

[STEP 4] DECRYPTION
Decrypted Message   : Hello Koushik Das

Decryption Time     : 0.2475 ms

[STEP 5] VERIFICATION
SUCCESS - Decrypted message matches original.
=====
```

5.2 Execution of 3desvsaes.py

When 3desvsaes.py is executed, the program performs two major operations:

1. Triple DES encryption performance benchmarking.
2. Comparative analysis between Triple DES and AES-128 encryption.

The program first encrypts plaintext messages of different sizes ranging from small to large inputs and measures the average encryption time across multiple iterations. This demonstrates how encryption time gradually increases with larger message sizes because Triple DES internally performs three DES operations for every encryption cycle.

The second stage compares Triple DES with AES-128 under identical CBC-mode conditions. Both algorithms encrypt large plaintext messages repeatedly, and their average execution times are displayed on the console. The results consistently show that AES performs encryption significantly faster than Triple DES because AES uses a more efficient algorithm structure and modern hardware optimization support.

Expected console output format:

```
text id="x2l7up"
=====
=====
      PERFORMANCE ANALYSIS OF TRIPLE DES
=====
=====

[3DES PERFORMANCE BENCHMARK]

Message Size      Encrypt Time
-----
100 bytes         0.1200 ms
1000 bytes        0.2500 ms
10000 bytes       1.8500 ms
100000 bytes      14.6000 ms

[3DES VS AES COMPARISON]

3DES Encryption Time : 15.4200 ms
AES Encryption Time  : 3.1800 ms

AES is approximately 4.85x faster than 3DES

=====
=====
```

Output:-(3DES vs AES)

```
● koushikdas@MAGNATE:~/Desktop/CNS Project 2026 [3DES]$ python 3DESvsAES.py
```

```
=====
PERFORMANCE ANALYSIS OF TRIPLE DES
=====
```

```
[3DES PERFORMANCE BENCHMARK]
```

Message Size	Encrypt Time
100 bytes	0.0995 ms
1000 bytes	0.1046 ms
10000 bytes	0.5011 ms
100000 bytes	4.3805 ms

```
[3DES VS AES COMPARISON]
```

```
3DES Encryption Time : 4.0498 ms
AES Encryption Time   : 0.1761 ms
AES is approximately 23.00x faster than 3DES
```

5.3 Execution of 3DES_SecurityTesting.py

When 3DES_SecurityTesting.py is executed, the program performs multiple security validation tests to verify the robustness and correctness of the Triple DES implementation.

The first test attempts to decrypt ciphertext using an incorrect secret key. The decryption operation fails successfully, demonstrating that only the correct key can recover the original plaintext. This confirms confidentiality protection.

The second test performs IV randomness validation. The same plaintext message is encrypted three times using the same key. Because a new random Initialization Vector is generated for each encryption operation, all ciphertext outputs differ from one another. This confirms correct implementation of CBC mode security.

The final stage performs edge-case testing using multiple categories of plaintext input including:

- single-character input,
- exact block-size input,
- special characters,
- Unicode characters,
- long plaintext messages

All test cases are encrypted and decrypted successfully without data corruption or execution failure, demonstrating the reliability and stability of the implementation.

Expected console interaction format:

```
text id="icupyr"
=====
      SECURITY TESTING OF TRIPLE DES
=====

Secure Random Key Generated Successfully.

[WRONG KEY TEST]
PASS — Wrong key decryption failed successfully.

[IV RANDOMNESS TEST]

Encrypted Output 1 : <Ciphertext 1>
Encrypted Output 2 : <Ciphertext 2>
Encrypted Output 3 : <Ciphertext 3>

PASS — Ciphertexts are different because of random IV.

[EDGE CASE TESTING]

Test Case          Status
-----
Single Character    PASS
```

Exactly 8 Bytes	PASS
Special Characters	PASS
Unicode Characters	PASS
Long Message	PASS

Output: - (Edge Case Testing)

```

koushikdas@MAGNATE:~/Desktop/CNS Project 2026 [3DES]$ python 3DES_SecurityTesting.py
=====
SECURITY TESTING OF TRIPLE DES
=====

Secure Random Key Generated Successfully.

[WRONG KEY TEST]
PASS – Wrong key decryption failed successfully.

[IV RANDOMNESS TEST]

Encrypted Output 1 : CqZGxyXGwmT+iV3g75p3eF4QUjfG3SYk9urk8yArIss=
Encrypted Output 2 : fIyrh0jLTUbs0laDURPUh2cKt9ZR1f6M1CjZodn9JBY=
Encrypted Output 3 : lgjT1GvkzjXnLqlYjRWUG3Jy+rvLXfKwttmkwfToKsI=

PASS – Ciphertexts are different because of random IV.

[EDGE CASE TESTING]

Test Case                      Status
-----
Single Character                PASS
Exactly 8 Bytes                PASS
Special Characters              PASS
Unicode Characters              PASS
Long Message                    PASS
=====

```

5.4 Interpretation of Results

The results obtained from all three programs confirm that the Triple DES implementation is functionally correct, operationally secure, and computationally reliable within the scope of educational and legacy cryptographic applications.

The 3DES_core.py implementation successfully performed complete encryption and decryption operations using Triple DES in CBC mode. The ciphertext generated during encryption appeared completely unreadable and structurally unrelated to the original plaintext, demonstrating effective confusion and diffusion properties of the Triple DES Feistel-based architecture.

The observed variation in ciphertext outputs across repeated executions confirmed the correct implementation of random Initialization Vectors (IVs). Even when identical plaintext messages were encrypted using the same key, the ciphertext outputs remained different because of CBC mode randomization. This behavior significantly improves confidentiality and prevents ciphertext pattern analysis.

The decryption stage successfully recovered the exact original plaintext in every execution test, confirming:

- correctness of the symmetric encryption-decryption cycle,
- integrity of Base64 encoding and decoding,
- correct implementation of padding and unpadding operations.

The benchmarking results demonstrated that Triple DES requires significantly more computation time than AES because Triple DES internally performs three DES operations for each encryption cycle. The AES comparison confirmed that AES is considerably faster and more efficient for large-scale modern cryptographic applications.

The security testing program further validated the robustness of the implementation through:

- wrong-key decryption failure,
- IV randomness testing,
- successful handling of Unicode characters,
- successful processing of long plaintext messages,
- correct encryption and decryption of special characters and exact block-size inputs.

Overall, the implementation successfully achieved all stated project objectives and demonstrated the practical working principles, security behavior, and operational characteristics of Triple DES encryption in modern Python-based cryptographic applications.

6. Conclusion

This project successfully designed and implemented a complete Secure Message Encryption and Decryption system using the Triple DES (3DES / TDEA) algorithm in Python. Three fully functional programs - 3DES_core.py, 3DESvsAES.py, and 3DES_SecurityTesting.py - were developed using the PyCryptodome library. Each program demonstrates a different aspect of the Triple DES operational workflow, including secure encryption and decryption, performance benchmarking, AES comparison, and security validation testing.

The implementation confirms that Triple DES reliably transforms arbitrary plaintext into unreadable ciphertext using a 168-bit key operating in CBC (Cipher Block Chaining) mode with a random Initialization Vector (IV). The corresponding decryption process accurately recovers the original plaintext without corruption or data loss. The use of Base64 encoding ensures that encrypted binary ciphertext can be safely displayed, transmitted, and stored as printable text within text-based communication systems.

From a security perspective, Triple DES provides a significant improvement over single DES by increasing effective key strength and making brute-force attacks computationally impractical for educational and legacy applications. The use of a random IV during every encryption operation ensures semantic security, causing identical plaintext messages to produce different ciphertext outputs during different executions. The wrong-key security test confirmed that unauthorized decryption attempts fail successfully, thereby maintaining confidentiality.

The performance analysis demonstrated that Triple DES is computationally slower than AES because Triple DES internally performs three DES operations for each encryption cycle. The AES comparison program confirmed that AES is significantly faster and more efficient for modern cryptographic systems because of its optimized structure and hardware acceleration support.

The security testing program successfully validated:

- wrong-key decryption resistance,
- IV randomness behavior,
- edge-case input handling,
- Unicode compatibility,
- stable encryption and decryption performance across multiple test cases.

Although Triple DES has been deprecated by NIST for modern large-scale applications because of limitations such as the 64-bit block size and Sweet32 vulnerability risks, understanding Triple DES remains highly important for educational purposes and for maintaining legacy banking and financial infrastructures that still depend on DES-based systems.

The project successfully achieved all stated objectives and demonstrated the practical implementation, operational workflow, security behavior, and performance characteristics of Triple DES encryption in modern Python-based cryptographic applications.

6.1 Learning Outcomes

1. Gained a strong understanding of symmetric-key cryptography, including block cipher operations, Triple DES architecture, CBC mode encryption, and Initialization Vector usage.
2. Developed practical experience using the PyCryptodome library in Python for implementing secure cryptographic operations using the DES3 cipher module.
3. Understood the security weaknesses of single DES and the rationale behind Triple DES as a stronger multi-stage enhancement over DES.
4. Learned the importance of secure encryption modes and understood why CBC mode is more secure than ECB mode because it prevents ciphertext pattern leakage.
5. Implemented secure random key generation and password-derived key generation using SHA-256 hashing techniques.
6. Understood how Base64 encoding allows binary ciphertext to be represented safely as readable ASCII text for display and storage.
7. Performed practical benchmarking and performance analysis to compare Triple DES encryption efficiency against AES-128.
8. Learned how random Initialization Vectors improve confidentiality by ensuring identical plaintexts encrypt into different ciphertext outputs.
9. Performed security validation using wrong-key testing, IV randomness testing, and edge-case input analysis.
10. Developed modular Python programs for encryption, benchmarking, and security testing following structured software engineering principles.

6.2 Future Improvements

While Triple DES remains operational in legacy environments, several enhancements would improve the security and functionality of this system in real-world deployments:

1. Migration to AES-256

Replacing Triple DES with AES-256 would improve both security and performance because AES supports larger block sizes, stronger cryptographic design, and hardware acceleration technologies such as AES-NI.

2. Authenticated Encryption (HMAC-SHA256)

Integrating HMAC-SHA256 alongside encryption would provide message integrity and authenticity verification in addition to confidentiality protection.

3. Secure Key Exchange Mechanism

Implementing Diffie-Hellman or RSA-based key exchange would demonstrate how two parties can securely exchange symmetric encryption keys over insecure communication channels.

4. File Encryption Support

Extending the implementation to support secure encryption and decryption of binary files, documents, images, and multimedia data.

5. Graphical User Interface (GUI)

Developing a GUI using Tkinter or PyQt would improve usability by allowing users to encrypt and decrypt messages through a visual interface.

6. Database Integration

The encrypted ciphertext and keys could be integrated with secure database storage systems for enterprise-level communication applications.

7. Network-Based Secure Communication

Future versions could implement socket-based encrypted client-server communication systems using Triple DES or AES encryption.

8. Hybrid Cryptographic System

Combining asymmetric encryption (RSA) with symmetric encryption (3DES or AES) would provide secure key exchange along with fast symmetric encryption performance.

9. Multi-User Authentication System

User authentication and access-control mechanisms could be added to restrict encryption and decryption operations to authorized users only.

10. Real-Time Message Encryption Application

The project can be expanded into a real-time encrypted messaging application for secure communication between multiple users over a network.

References

- [1] I. Neil, CompTIA Security+ SY0-701 Certification Guide, 3rd ed., Packt Publishing, 2023.

- [2] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed., Hoboken, NJ: Pearson Education, 2022.

- [3] National Institute of Standards and Technology (NIST), Recommendation for Triple Data Encryption Algorithm (TDEA) Block Cipher, NIST Special Publication 800-67 Rev. 2, Nov. 2017. [Online]. Available: [NIST SP 800-67 Rev 2](https://doi.org/10.6028/NIST.SP.800-67r2?utm_source=chatgpt.com)

- [4] National Institute of Standards and Technology (NIST), Transitions: Recommendation for Transitioning the Use of Cryptographic Algorithms and Key Lengths, NIST Special Publication 800-131A Rev. 2, Mar. 2019. [Online]. Available: [NIST SP 800-131A Rev 2](https://doi.org/10.6028/NIST.SP.800-131Ar2?utm_source=chatgpt.com)

- [5] K. Bhargavan and G. Leurent, “On the Practical (In-)Security of 64-bit Block Ciphers: Collision Attacks on HTTP over TLS and OpenVPN,” in Proceedings of the ACM Conference on Computer and Communications Security (CCS 2016), Vienna, Austria, 2016.

- [6] PyCryptodome Documentation] (https://pycryptodome.readthedocs.io?utm_source=chatgpt.com), “PyCryptodome Cryptographic Library for Python,” 2024.

- [7] B. Schneier, Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd ed., New York: John Wiley & Sons, 1996.

- [8] National Institute of Standards and Technology (NIST), Data Encryption Standard (DES), FIPS PUB 46-3, Oct. 1999 (withdrawn 2005).